SOFTWARE REQUIREMENTS SPECIFICATION

Food Delivery Management Platform

IEEE-Style SRS Document

Team Members:

Deepak Singh - 202401100200123

Deepak Nimesh - 202401100200122

Deepak Maurya - 202401100200121

Deependra Ahirwar – 202401100200124

B.Tech. CSE Department

KIET GROUP OF INSTITUTIONS

2026-2027

PROJECT VERISON – 1.0

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to describe what the Food Delivery Management Platform should do and how it is expected to behave. The document acts as a common reference for the project team, guide, developers, testers, and future maintainers. Instead of focusing on source code, it explains the required features, users, interfaces, constraints, and quality expectations of the proposed system.

1.2 Scope

The Food Delivery Management Platform is a web-based system that brings customers, restaurants, delivery partners, and administrators together. Customers can find restaurants, check menus, place orders, make payments, follow delivery progress, and share ratings or reviews. Restaurants can manage their menus, availability, offers, and incoming orders. Delivery partners can manage availability and assigned deliveries. Administrators can supervise users, restaurants, orders, payments, complaints, and general platform activity.

1.3 Definitions, Acronyms & Abbreviations

Term	Meaning
SRS	Software Requirements Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
API	Application Programming Interface
UI	User Interface
GPS	Global Positioning System
ETA	Estimated Time of Arrival
RBAC	Role-Based Access Control
JWT	JSON Web Token
DFD	Data Flow Diagram
ER	Entity Relationship

1.4 References

The document follows the organization and spirit of the IEEE 830-style Software Requirements Specification format shown in the provided template. Detailed implementation should also refer to the official documentation of the technologies and third-party services selected during development.

1.5 Overview

The next section gives an overall view of the product and its users. Section 3 contains the specific requirements, including interfaces, functional requirements, performance, design constraints, and software quality attributes. The final section contains supporting information such as a contents/index summary and appendices.

2. Overall Description

2.1 Product Perspective

The platform can be viewed as a central service connecting four main user groups. A customer creates an order, the restaurant prepares it, and a delivery partner completes the delivery. The administrator acts as the operational layer that keeps the platform organized. The system may also communicate with external services such as a payment gateway, map/location service, and notification service.

2.1.1 System Interfaces

The system interface consists of role-based web screens such as the customer home page, restaurant dashboard, delivery dashboard, and administrator panel. These screens communicate with backend APIs and should present information in a simple and consistent way.

2.1.2 User Interfaces

- Customer: restaurant search, menu, cart, checkout, tracking, history, and reviews.
- Restaurant: profile, menu management, order processing, offers, and earnings.
- Delivery Partner: availability, assignments, pickup/delivery details, and delivery history.
- Administrator: account management, monitoring, complaints, reports, and system controls.

2.1.3 Hardware Interfaces

No special hardware is mandatory for the basic system. Users need an internet-enabled computer or mobile device. Delivery-related location features may use the GPS capability of a supported mobile device.

2.1.4 Software Interfaces

The application will communicate with a database through the backend service and may connect to payment, map/location, and notification APIs. The exact providers can be selected during implementation.

2.1.5 Communications Interfaces

Client-server communication should use HTTPS. REST-style APIs can exchange structured JSON data. External services should be accessed through their documented and authenticated interfaces.

2.1.6 Memory Constraints

The system should not assume unlimited memory or storage. Uploaded images, logs, and historical records should be managed carefully. Database indexing and suitable cleanup or archival policies should be considered as the amount of data increases.

2.1.7 Operations

Normal operations include user registration, restaurant/menu updates, order processing, payment handling, delivery status updates, notifications, and administrative monitoring. Backup, monitoring, and recovery procedures should be included before the system is used in a production environment.

2.1.8 Site Adaptation Requirements

The interface should adapt to desktop and mobile screen sizes. Operational rules such as delivery areas, available payment methods, offers, taxes, and service charges should be configurable rather than hard-coded.

2.2 Product Functions

Module	Main Functions
Customer	Register/login, browse restaurants, search/filter, view menu, manage cart, manage address, place/cancel order, payment, tracking, history, rating/review, notifications.
Restaurant	Register/login, profile management, menu CRUD, availability, receive orders, accept/reject, preparation status, offers, history, earnings.
Delivery Partner	Register/login, profile, availability, assignment, pickup details, delivery location, status updates, history, earnings.
Administrator	Authentication, account management, restaurant management, order/payment monitoring, complaints, reviews, offers, reports, suspension/activation.

2.3 User Characteristics

The platform is intended for users with normal smartphone or web browsing skills. Customers should not need technical knowledge. Restaurant and delivery users need basic familiarity with dashboards and mobile interfaces. Administrators require more training because their actions can affect multiple users and orders.

2.4 Constraints

- The platform depends on a stable internet connection.
- Payment and map/location features depend on third-party services.
- Delivery accuracy depends partly on GPS and location data.
- Restaurant and delivery partner availability affects fulfilment.
- Privacy, security, and applicable legal requirements must be respected.

2.5 Assumptions and Dependencies

It is assumed that users provide reasonably accurate information, restaurants keep menus and availability updated, and delivery partners update their availability honestly. The project also depends on the uptime and API policies of selected third-party providers.

2.6 Apportioning of Requirements

The first release should focus on the basic end-to-end journey: authentication, restaurant/menu browsing, cart, order placement, payment status, restaurant processing, delivery assignment, and order completion. More advanced features such as AI recommendations, route optimization, loyalty programs, and predictive analytics can be treated as later releases.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The interface should feel straightforward rather than overloaded. Important actions such as Add to Cart, Place Order, Accept Order, and Update Delivery Status should be easy to find. Forms should show useful validation messages instead of only displaying generic errors.

3.1.2 Hardware Interfaces

The application should work with standard laptops, desktops, tablets, and smartphones. Where delivery tracking is supported, the mobile device may provide location information with user permission.

3.1.3 Software Interfaces

The backend should expose controlled APIs to the frontend. The database should store user, restaurant, menu, cart, order, payment, delivery, review, notification, and complaint information. Third-party services should be integrated through documented APIs.

3.1.4 Communication Interfaces

HTTPS should be used for production communication. API responses should use consistent status codes and structured data. Authentication information must not be transmitted through insecure channels.

3.2 Specific Requirements

3.2.1 Sequence Diagram

A typical order sequence starts when a customer selects food and submits the cart. The backend validates the customer, address, restaurant, items, and total. Payment is then processed or the COD option is recorded. The restaurant receives the order and accepts or rejects it. Once the food is ready, a delivery partner is assigned. The partner picks up the order, updates the delivery status, and finally marks the order as delivered.

Step	Actor	System Response
1	Customer	Selects restaurant, food items, address, and payment method.
2	System	Validates cart, user, restaurant status, and order details.
3	Payment Service	Returns payment success/failure when online payment is used.
4	Restaurant	Accepts/rejects and prepares the order.
5	Delivery Partner	Accepts assignment, picks up order, and updates delivery status.
6	Customer	Receives status updates and confirms the completed delivery experience through rating/review.

3.2.2 Classes for Classification of Specific Requirements

The following requirements are grouped so that they can later be mapped to modules, use cases, and test cases.

ID	Requirement	Priority
FR-001	Customer shall be able to register and log in securely.	High
FR-002	Customer shall be able to browse and filter active restaurants.	High
FR-003	Customer shall be able to view restaurant menus and item availability.	High
FR-004	Customer shall be able to add, update, and remove cart items.	High
FR-005	Customer shall be able to place an eligible order.	High
FR-006	Customer shall be able to view order status and history.	High
FR-007	Customer shall be able to rate and review completed orders.	Medium
FR-008	Restaurant shall be able to add, edit, delete, and disable food items.	High
FR-009	Restaurant shall be able to accept/reject and process orders.	High
FR-010	Delivery partner shall be able to manage availability and assignments.	High
FR-011	Delivery partner shall be able to update authorized delivery states.	High
FR-012	Administrator shall be able to manage platform accounts and operational records.	High
FR-013	System shall record online payment success/failure and refund status.	High
FR-014	System shall generate notifications for important order events.	High

3.3 Performance Requirements

- Normal browsing and form operations should feel responsive to users under the expected project workload.
- The system should support multiple users at the same time without mixing their sessions or data.
- Database queries used frequently, such as restaurant search and order lookup, should be indexed appropriately.
- External service delays should not unnecessarily block unrelated application operations.

3.4 Design Constraints

- The architecture should keep frontend, backend, database, and external integrations reasonably separated.
- Security-sensitive operations must be performed on the server rather than trusted only to client-side validation.
- Order state changes should follow a controlled lifecycle.
- The design should allow future features without rewriting the complete system.

3.5 Software System Attributes

3.5.1 Reliability

The system should preserve order and payment consistency even when an external service fails. Important operations should return clear success or failure states. Database backups and recovery procedures should be considered for important deployment environments.

3.5.2 Availability

The deployed application should be monitored so that service failures can be detected. Health checks, logging, and appropriate restart or recovery mechanisms should be used. Third-party service outages should be handled gracefully where possible.

3.5.3 Security

- Passwords must never be stored as plain text.
- Role-based authorization must restrict protected operations.
- Production API communication should use HTTPS/TLS.
- Input should be validated to reduce injection and malformed-data risks.
- The system should apply appropriate protections against XSS and CSRF based on its authentication design.
- Payment credentials should be handled by the payment provider wherever practical.
- Administrative and security-sensitive actions should be logged appropriately.

3.5.4 Maintainability

The codebase should be organized into understandable modules, with consistent naming, API documentation, configuration management, and version control. A new developer should be able to understand the main flow without depending entirely on one team member.

3.6 Other Requirements

The system should support basic data privacy practices, meaningful error messages, and accessible user interfaces. Important records such as orders and payments should remain traceable. The platform should also keep terminology consistent across customer, restaurant, delivery, and administrator screens.

4. Supporting Information

4.1 Table of Contents and Index

- 1. Introduction — Purpose, Scope, Definitions, References, Overview
- 2. Overall Description — Product Perspective, Interfaces, Product Functions, Users, Constraints, Assumptions
- 3. Specific Requirements — External Interfaces, Functional Requirements, Performance, Design Constraints, Quality Attributes
- 4. Supporting Information — Table of Contents/Index and Appendices

4.2 Appendices

Appendix A — Order Lifecycle

Placed → Payment Confirmed → Restaurant Accepted → Food Preparing → Ready for Pickup → Delivery Assigned → Picked Up → Out for Delivery → Delivered.

Possible exception states include Payment Failed, Restaurant Rejected, Cancelled, Delivery Failed, Refund Initiated, and Refunded. Only valid and authorized state transitions should be allowed.

Appendix B — Main Data Entities

Entity	Purpose
User	Common account and role information.
Customer	Customer-specific profile and preferences.
Restaurant	Restaurant profile and operating status.
FoodItem	Menu item, price, description, and availability.
Cart / CartItem	Temporary customer selections before checkout.
Address	Saved customer delivery addresses.
Order / OrderItem	Purchase and its line items.
Payment	Payment method, provider reference, amount, and status.
Delivery / DeliveryPartner	Delivery assignment, partner, and delivery status.
Review / Rating	Customer feedback for eligible completed orders.
Offer	Discount or promotional rules.
Notification	User-facing event messages.
Complaint	Customer or partner complaint and resolution details.

Appendix C — Proposed Technology Stack

Frontend: React.js. Backend: Node.js with Express.js. Database: MongoDB. Authentication: JWT or another secure mechanism. API: REST. Payment: secure third-party payment gateway. Maps/Location: suitable mapping and geolocation API. Version Control: Git/GitHub. Deployment: cloud-based hosting. These are proposed choices and can be adjusted during implementation if project constraints require it.

Appendix D — Basic Acceptance Checklist

- Customer can register, log in, browse restaurants, and view menus.
- Customer can add items to a cart and place an order.
- Restaurant can receive, accept/reject, and prepare an order.
- Delivery partner can receive an assignment and complete a delivery.
- Order status is visible to the customer.
- Payment status is recorded correctly.
- Administrator can manage important platform records.
- Invalid or unauthorized actions are rejected safely.

— End of SRS Document —